Short Course in MATLAB

Lecture 4

Amit Harlev

Inverting colors

What if we want to invert the colors in the image?

Inverting colors means replacing white pixels with black pixels, light gray pixels with dark gray pixels, and so on.

Since brightness is represented by a number between `0` and `255`, we replace each pixel's brightness `x` with `255 - x`.

Inverting colors

What if we want to invert the colors in the image?

Inverting colors means replacing white pixels with black pixels, light gray pixels with dark gray pixels, and so on.

Since brightness is represented by a number between `0` and `255`, we replace each pixel's brightness `x` with `255 - x`.

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Inverting colors: Vectorized operations

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Can we do this more efficiently?

Inverting colors: Vectorized operations

```
inverted_flower = flower;

for row = 1:size(flower, 1)
    for col = 1:size(flower, 2)
        inverted(row, col) = 255 - flower(row, col);
    end
end

imshow(inverted);
```

Can we do this more efficiently?

Just like comparison operators, MATLAB let's us apply basic math operations to entire arrays.

These are called *vectorized operations*.

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

What will we get if we do `a * b`?

Vectorized operations

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a + 4; % [5 6; 7 8]
a - 4; % [-3 -2; -1 0]

a + b; % [2 2; 3 5]
```

What will we get if we do `a * b`?

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

By default, `*`, `/`, and `^` will be treated as matrix operations, not element-by-element operations.

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a .* b; % [1 0; 0 4]
a .^ 2; % [1 4; 9 16]
```

What is `a .^ b`?

Vectorized operations

If you want the element-by-element operations, you add a dot for the following operations:

- multiplication is `.*`
- division is `./`
- exponentiation is `.^`

```
a = [1 2; 3 4];
b = [1 0; 0 1];

a .* b; % [1 0; 0 4]
a .^ 2; % [1 4; 9 16]
```

What is `a .^ b`?

`[1 1; 1 4]`

Back to inverting colors

What is the most efficient way to invert the colors for the flower image?

Back to inverting colors

What is the most efficient way to invert the colors for the flower image?

```
inverted_flower = 255 - flower;
imshow(inverted_flower);
```

Color images in MATLAB

```
dog = imread("dog.jpg");

size(dog); % 1440 by 960 by 3
```

Color images in MATLAB

```
dog = imread("dog.jpg");

size(dog); % 1440 by 960 by 3
```

`dog` is a 3d array where

- `dog(:, :, 1)` is the amount of red in each pixel.

Color images in MATLAB

```
dog = imread("dog.jpg");

size(dog); % 1440 by 960 by 3
```

`dog` is a 3d array where

- `dog(:, :, 1)` is the amount of red in each pixel.
- `dog(:, :, 2)` is the amount of green in each pixel.

Color images in MATLAB

```
1 dog = imread("dog.jpg");
2
3 size(dog); % 1440 by 960 by 3
```

`dog` is a 3d array where

- `dog(:, :, 1)` is the amount of red in each pixel.
- `dog(:, :, 2)` is the amount of green in each pixel.
- `dog(:, :, 3)` is the amount of blue in each pixel.

Visualizing each channel: Red

```
dog = imread("dog.jpg");
red_dog = dog;
red_dog(:, :, 2:3) = 0;
imshow(red_dog);
```

Visualizing each channel: Red

```
dog = imread("dog.jpg");
red_dog = dog;
red_dog(:, :, 2:3) = 0;
imshow(red_dog);
```

Visualizing each channel: Red

```
dog = imread("dog.jpg");
red_dog = dog;
red_dog(:, :, 2:3) = 0;
imshow(red_dog);
```

Recall that `2:3` is the same as the array `[2 3]`.

So we could instead have written `red_dog(:, :, [2 3]) = 0`.

Visualizing each channel: Green

```
dog = imread("dog.jpg");
green_dog = dog;
green_dog(:, :, [1 3]) = 0;
imshow(green_dog);
```

Visualizing each channel: Green

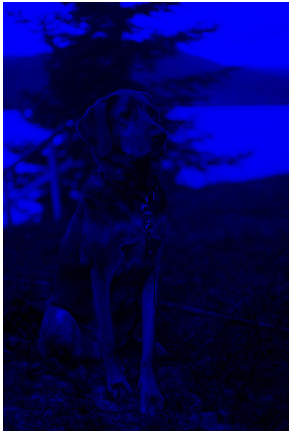
```
dog = imread("dog.jpg");
green_dog = dog;
green_dog(:, :, [1 3]) = 0;
imshow(green_dog);
```

Visualizing each channel: Blue

```
dog = imread("dog.jpg");
blue_dog = dog;
blue_dog(:, :, [1 3]) = 0;
imshow(blue_dog);
```

Visualizing each channel: Blue

```
dog = imread("dog.jpg");
blue_dog = dog;
blue_dog(:, :, [1 3]) = 0;
imshow(blue_dog);
```



Combining channels: Red + Green

```
1 imshow(red_dog + green_dog);
```

Combining channels: Red + Green

```
1 imshow(red_dog + green_dog);
```

Combining channels: Red + Green + Blue

```
1 imshow(red_dog + green_dog + blue_dog);
```

Combining channels: Red + Green + Blue

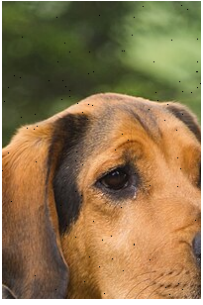
```
1 imshow(red_dog + green_dog + blue_dog);
```

A new problem

Imagine your image got corrupted and there are now a bunch of randomly spread out black pixels.

A new problem

Imagine your image got corrupted and there are now a bunch of randomly spread out black pixels.



How might we fix this?

Logical indexing for arrays

```
1 a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 1: Use a for loop and an if statement

```
for i = 1:length(a)
    if a(i) < 0
        a(i) = a(i)^2;
    end
end
```

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 2: Index the negative positions directly

```
a([1 2 5]) = a([1 2 5]).^2;
```

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 3: Use logical indexing

```
a([true true false false true]); % [-2 -3 -1]
```

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 3: Use logical indexing

```
a([true true false false true]); % [-2 -3 -1]
```

This is equivalent to `a([1 2 5])`.

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 3: Use logical indexing

```
a([true true false false true]); % [-2 -3 -1]
```

This is equivalent to `a([1 2 5])`. So we can do:

```
a([true true false false true]) = a([true true false false true]).^2;
```

Logical indexing for arrays

```
a = [-2 -3 3 6 -1];
```

Goal: Square only the negative entries of the array.

Approach 3: Use logical indexing

```
a([true true false false true]); % [-2 -3 -1]
```

This is equivalent to `a([1 2 5])`. So we can do:

```
a([true true false false true]) = a([true true false false true]).^2;
```

But that means we can also do:

```
a(a < 0) = a(a < 0).^2; % best solution
```

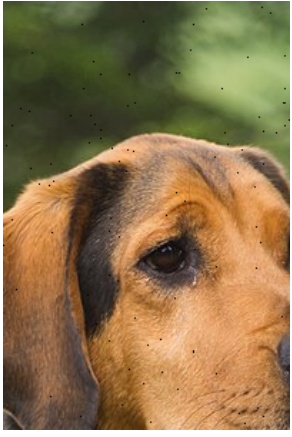
Back to the corrupted image

We can use logical indexing to find all the black pixels. How?

Back to the corrupted image

We can use logical indexing to find all the black pixels. How?

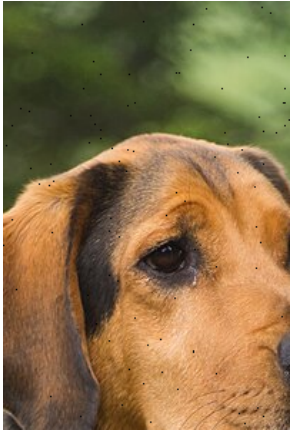
```
no_red = corrupted(:, :, 1) == 0;
no_green = corrupted(:, :, 2) == 0;
no_blue = corrupted(:, :, 3) == 0;
```



Back to the corrupted image

We can use logical indexing to find all the black pixels. How?

```
no_red = corrupted(:, :, 1) == 0;
no_green = corrupted(:, :, 2) == 0;
no_blue = corrupted(:, :, 3) == 0;
black_pixels = no_red & no_green & no_blue;
```

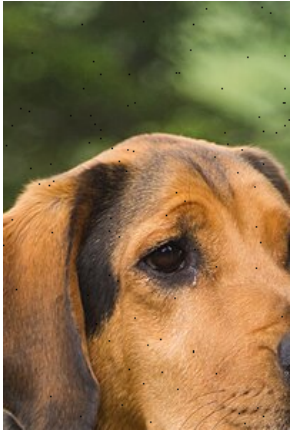


Back to the corrupted image

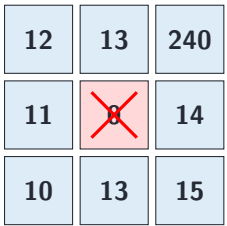
We can use logical indexing to find all the black pixels. How?

```
no_red = corrupted(:, :, 1) == 0;
no_green = corrupted(:, :, 2) == 0;
no_blue = corrupted(:, :, 3) == 0;
black_pixels = no_red & no_green & no_blue;
```

Then we can blur at each of these black pixels to "fix" the image.

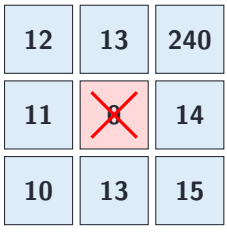


Mean blur without center pixel



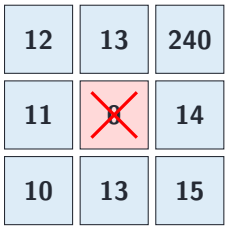
Mean blur without center pixel

1. **Add the eight neighboring values:**



$$12 + 13 + 240 + 11 + 14 + 10 + 13 + 15 = 328$$

Mean blur without center pixel



Ignore the center

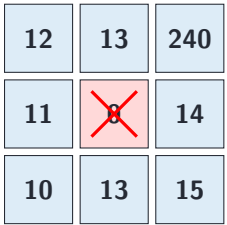
1. **Add the eight neighboring values:**

$$12 + 13 + 240 + 11 + 14 + 10 + 13 + 15 = 328$$

2. **Divide by the number of neighbors:**

$$\text{mean} = \frac{328}{8} = \mathbf{41}$$

Mean blur without center pixel



Ignore the center

1. **Add the eight neighboring values:**

$$12 + 13 + 240 + 11 + 14 + 10 + 13 + 15 = 328$$

2. **Divide by the number of neighbors:**

$$\text{mean} = \frac{328}{8} = \mathbf{41}$$

3. **Replace the center pixel with the mean:**

$$\begin{bmatrix} 12 & 13 & 240 \\ 11 & \boxed{\mathbf{41}} & 14 \\ 10 & 13 & 15 \end{bmatrix}$$

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Some useful notes:

- `imread` returns an array of type uint8: 8-bit integers. See demo.

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Some useful notes:

- `imread` returns an array of type uint8: 8-bit integers. See demo.
- Can convert to doubles using `double()`
- Can convert back at the end before updating value in image using `uint8()`

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Some useful notes:

- `imread` returns an array of type uint8: 8-bit integers. See demo.
- Can convert to doubles using `double()`
- Can convert back at the end before updating value in image using `uint8()`
- `numel(array)` returns the number of elements in the array.

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Some useful notes:

- `imread` returns an array of type uint8: 8-bit integers. See demo.
- Can convert to doubles using `double()`
- Can convert back at the end before updating value in image using `uint8()`
- `numel(array)` returns the number of elements in the array.
- You need to apply the mean blur to each channel individually.

Activity: remove the corrupted pixels

Our goal is to write the following function and then fix the corrupted image.

```
function result = maskedMeanBlur(img, mask)
% Applies an eight-neighbor mean blur only where mask is true.
% Assumes img is an RGB image with exactly three channels.
```

Some useful notes:

- `imread` returns an array of type uint8: 8-bit integers. See demo.
- Can convert to doubles using `double()`
- Can convert back at the end before updating value in image using `uint8()`
- `numel(array)` returns the number of elements in the array.
- You need to apply the mean blur to each channel individually.
- Blurs near the boundary should use just the pixels that exist!